

# A Tale of Five Decoders.

David Ponarovsky

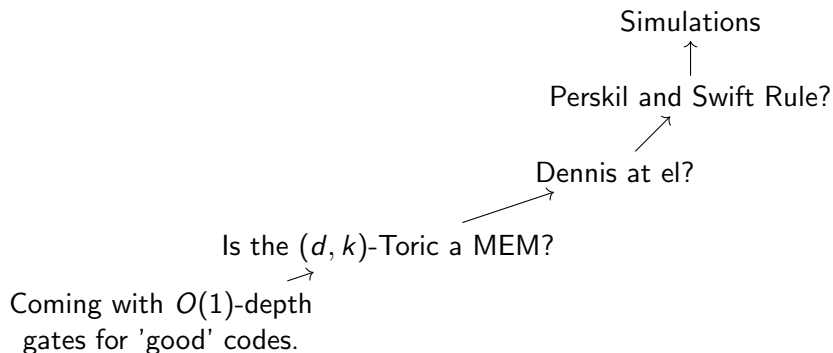
November 4, 2025

# Introduction

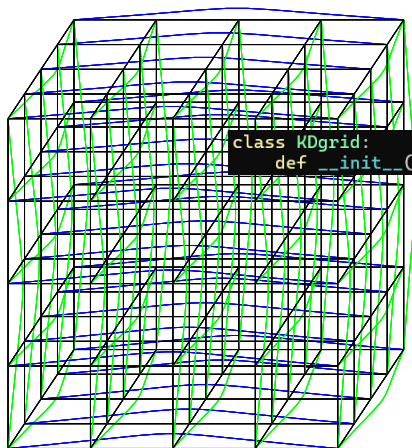
## Today:

- ▶ Recap, and where we stood before the summer.
- ▶ Simulations Results.
- ▶ Where we heading.

# Map

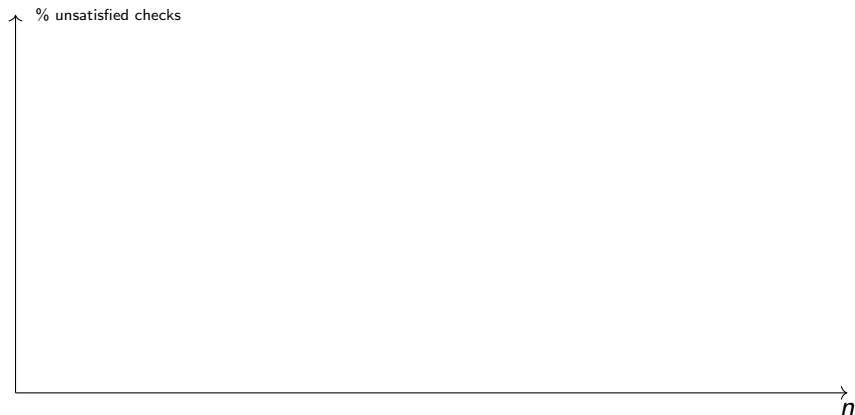


# The Code



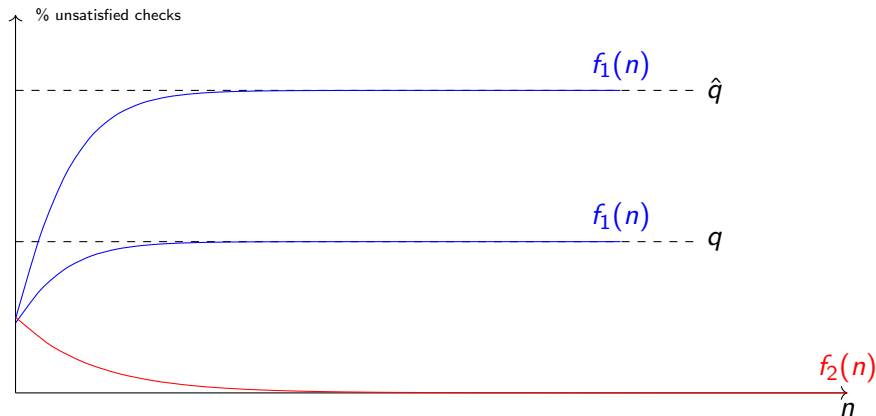
```
class KDgrid:  
    def __init__(self, n, k, celldim=2, checksdim=0,
```

# How to Read



side-length :[8, 16, 20, 30, 30, 30, 30]  
grid dim :[3, 3, 3, 3, 3, 3, 3]  
error rates :[0.001]  
error accumulation :False

# Ideal Result.



colors

In red, No accumulation of errors. In blue decoding in the presence of noise.

# Majority

## Alg.

Each bit looks at its neighbour checks, if the majority of them is unsatisfied, then it flip itself.

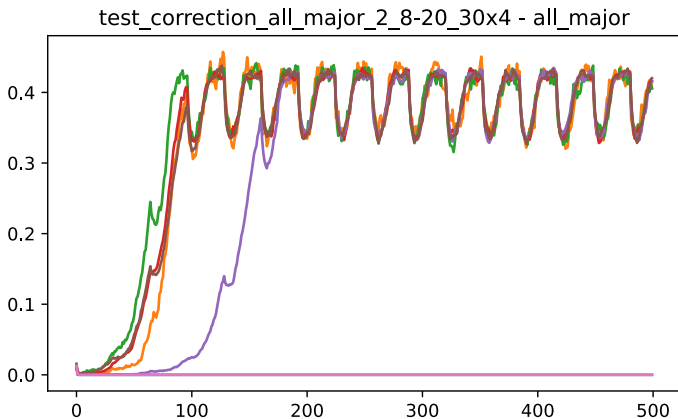
## Justification.

Was suggested in Dennis.

## Bottom line.

- ▶ Fails to correct, even in the absence of noise.
- ▶ Collisions.
- ▶ Gives better result when requiring  $(\frac{1}{2} + \mu)$ -majority.

# Majority

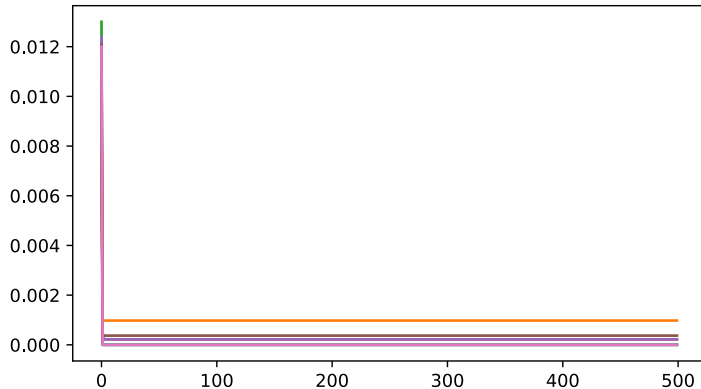


side-length :[8, 16, 20, 30, 30, 30, 30]  
grid dim :[3, 3, 3, 3, 3, 3, 3]  
error rates :[0.001]  
error accumulation :False



# Majority

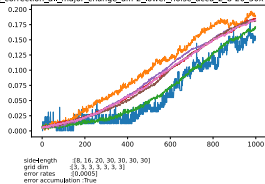
test\_correction\_all\_major\_change\_diff-2\_2\_8-20\_30x4 - all\_major



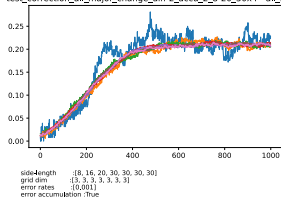
side-length :[8, 16, 20, 30, 30, 30, 30]  
grid dim :[3, 3, 3, 3, 3, 3, 3]  
error rates :[0.001]  
error accumulation :False

# Majority

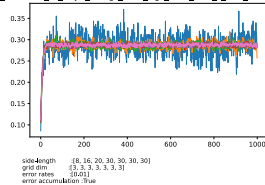
test\_correction\_all\_major\_change\_diff-2\_lower\_noise\_accu\_2\_8-20\_30x4 - all\_major



test\_correction\_all\_major\_change\_diff-2\_accu\_2\_8-20\_30x4 - all\_major



test\_correction\_all\_major\_change\_diff-2\_high\_noise\_accu\_2\_8-20\_30x4 - all\_major



# Colors

## Alg.

At the initialization, We divide the bits into subsets (colors). Bits of the same color don't share common check. Then, at the correction round  $i$ th only bits associated with the color  $i \bmod \mathbf{COLORS}$  apply a correction.

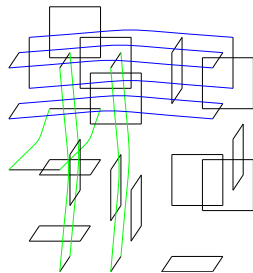
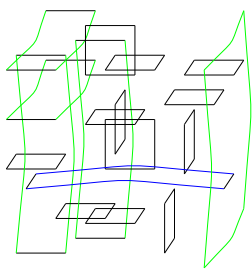
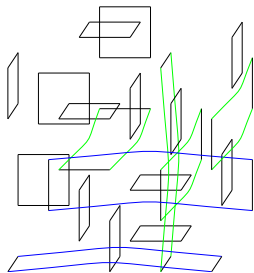
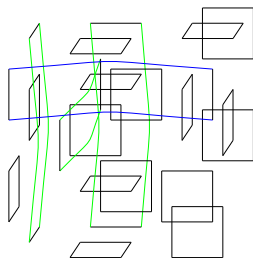
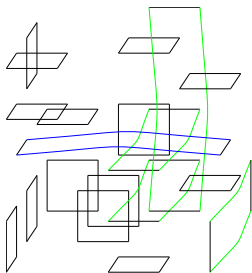
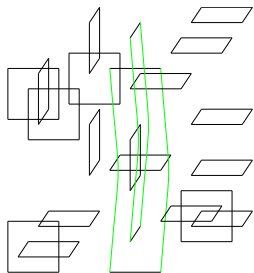
## Justification.

1. No collisions, means that bits don't wait until they can read the syndrome.
2. The independence might help when coming to the math.

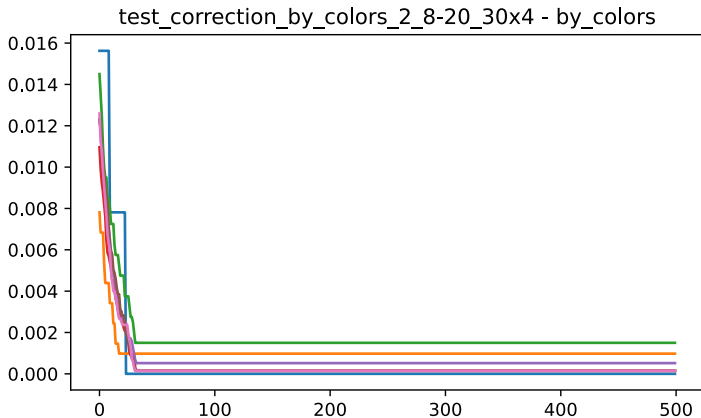
## Bottom line.

- ▶ I took 32 colors, and sample a random coloring.
- ▶ Fails to correct even in the absence of noise.
- ▶ Not tested yet when taking  $(\frac{1}{2} + \mu)$ -majority.

# Colors



# Colors



side-length :[8, 16, 20, 30, 30, 30, 30]  
grid dim :[3, 3, 3, 3, 3, 3, 3]  
error rates :[0.001]  
error accumulation :False

# Picking Random Pair

## Alg.

Each bit pick two random neighbour checks, and then check if both of them are not satisfied. If so, then flip itself.

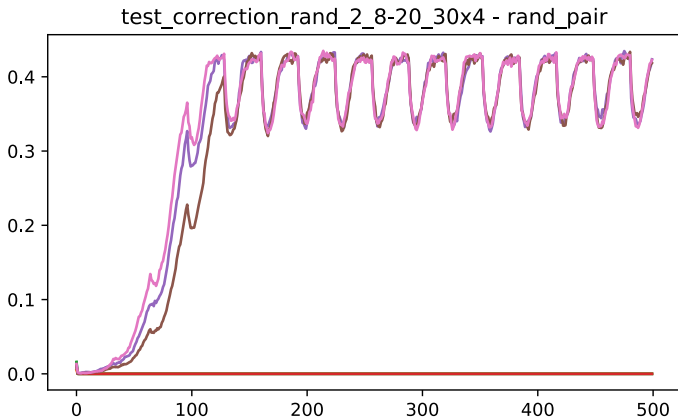
## Justification.

With good probability, no too many collisions. Yet, all the bits participate in each round.

## Bottom line.

- ▶ Fails to correct even in the absence of noise.
- ▶ Can we find an analog to  $(\frac{1}{2} + \mu)$ -majority? Maybe picking 3?

# Picking Random



side-length :[8, 16, 20, 30, 30, 30, 30]  
grid dim :[3, 3, 3, 3, 3, 3, 3]  
error rates :[0.001]  
error accumulation :False

# Swift Rule

## Alg.

We define a north vector  $\mathbf{1} = (1, 1, 1..1)$ , Then each vertex looks only on the cells on its neighbourhood such that they are positive relative to  $\mathbf{1}$ .

Then, the vertex suggest a correction over those cells, that decreases the local syndrome.

## Justification.

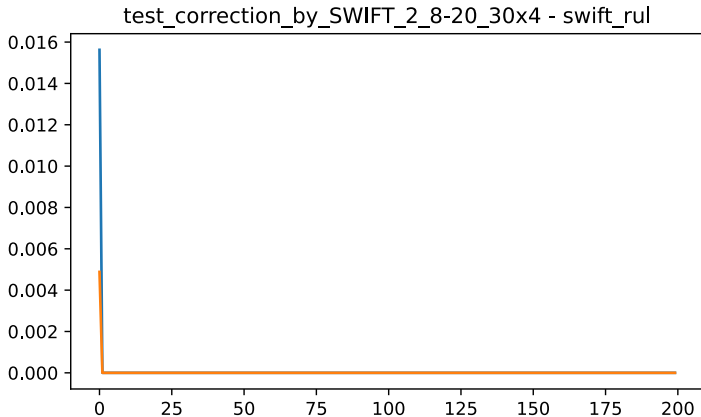
1. In the absence of noise, was proven to work.
2. Considered a generalization of Toom's rule.

## Bottom line.

- ▶ Works in the absence of noise.
- ▶ Simulation are too slow for side length of  $n > 16$
- ▶ Don't have yet results for accumulating setting.



# SWIFT



side-length :[8, 16, 20, 30, 30, 30, 30]  
grid dim :[3, 3, 3, 3, 3, 3, 3]  
error rates :[0.001]  
error accumulation :False

## The Bottleneck

Running a SWIFT simulation requires more than weeks.

```
davidpo+ 690 100 1.9 2064736 158696 pts/0 R+ Oct28 4968:07 python ./tests.py
davidpo+ 691 99.9 3.0 2152332 245768 pts/0 R+ Oct28 4968:02 python ./tests.py
davidpo+ 692 99.9 7.6 2520108 609584 pts/0 R+ Oct28 4967:52 python ./tests.py
davidpo+ 741 99.9 7.6 2522152 611480 pts/0 R+ Oct28 4967:42 python ./tests.py
davidpo+ 780 99.9 7.7 2526108 615532 pts/0 R+ Oct28 4967:32 python ./tests.py
root 818 0.0 0.0 3064 896 ? Ss Oct28 0:00 /init
root 820 0.0 0.0 3080 1024 ? S Oct28 0:01 /init
davidpo+ 822 0.0 0.0 10480 7116 pts/4 Ss Oct28 0:00 -zsh
davidpo+ 865 99.9 8.1 2564004 653520 pts/0 R+ Oct28 4967:21 python ./tests.py
root 2272 0.0 0.0 2064 896 ? Ss Oct28 0:00 /init
```

# Max Color

## Alg.

We change the setting. We back to the setting that allows a non-nosy computation aside, yet we would like to limit the communication to constant number of bits.

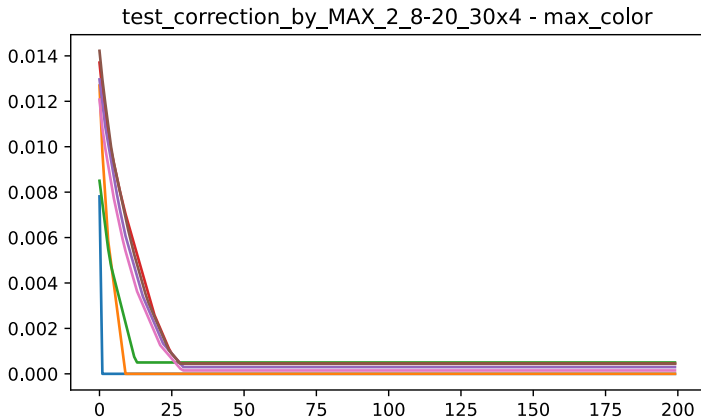
## Justification.

1. We don't have works about bounded communication fault tolerance.
2. From mathematical point of view, if the number of colors is  $c$ , then we can handle a fraction of  $1/c$  of the dirty bits.

## Bottom line.

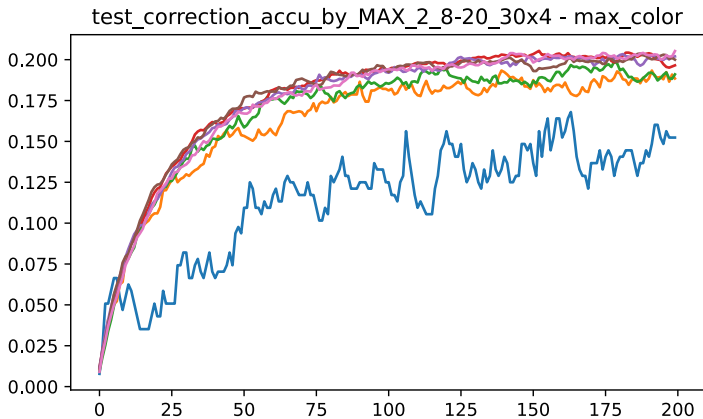
- ▶ Works in the absence of noise.
- ▶ 'Simulatable'.
- ▶ Should be tried when vertices correct according to  $(\frac{1}{2} + \mu)$  majority.

# Max Color



side-length :[8, 16, 20, 30, 30, 30, 30]  
grid dim :[3, 3, 3, 3, 3, 3, 3]  
error rates :[0.001]  
error accumulation :False

# Max Color



side-length :[8, 16, 20, 30, 30, 30, 30]  
grid dim :[3, 3, 3, 3, 3, 3, 3]  
error rates :[0.001]  
error accumulation :True

## So What's Next?

1. Moving forward with the experiential work?
  - ▶ More experiments, higher statistical standard?
  - ▶ Accelerate the decoder? Shifting to  $c/c++$ ? Parallelize it with GPUs/FPGA? Do we have budget? Publishing the source as library?
  - ▶ Publishing 'A Tale of Five Decoders'?
2. Clearly, right now, we don't have a memory code. Yet, we do have enough for writing inelegantly on it. Should we write a review paper?
3. What about Bounded-communication Fault Tolerance?
4. Moving to the  $B$  stage, can we do it by December?